

INF111
Travail Pratique 1:
Système de gestion de bibliothèque
Auteur : Asma BELLILI
Groupes : INF111_80 , INF111_03
Été 2026

Contexte de l'application

Ce travail pratique a pour objectif de réaliser une application de gestion de bibliothèque. À travers cette application, plusieurs utilisateurs pourront interagir avec un système informatique afin d'effectuer différentes opérations liées à la gestion des livres et des emprunts.

Un utilisateur devra notamment pouvoir consulter les livres disponibles, emprunter un livre, retourner un livre déjà emprunté, réserver un livre indisponible ainsi que consulter les informations associées à ses emprunts.

L'application devra également assurer la gestion des utilisateurs inscrits, des emprunts en cours, des emprunts terminés, des réservations ainsi que des pénalités appliquées en cas de retard.

Objectifs

Le travail demandé permettra de mettre en pratique plusieurs notions importantes du cours, notamment :

- les classes et les objets ;
- l'encapsulation ;
- les constructeurs ;
- les énumérations ;
- l'héritage ;
- les classes abstraites ;
- les interfaces ;
- les collections Java ;

Organisation du travail

Afin de réaliser ce travail pratique de manière progressive, le développement de l'application sera divisé en trois parties.

Dans la première partie, vous devrez mettre en place les classes nécessaires à la gestion de la bibliothèque. Cette étape consiste principalement à construire le modèle objet de l'application en créant les différentes entités du système, par exemple les livres, les utilisateurs, les emprunts, les réservations ainsi que les structures permettant de les stocker et de les gérer.

Dans la deuxième partie, vous devrez implémenter le traitement des demandes envoyées par les utilisateurs. Lorsqu'un utilisateur effectue une opération, le programme devra analyser la demande, vérifier que toutes les conditions nécessaires sont respectées, puis exécuter l'action appropriée. Par exemple, lors d'une demande d'emprunt, le système devra vérifier que l'utilisateur existe, que le livre est disponible et que la limite d'emprunts de l'utilisateur n'est pas dépassée avant de créer l'emprunt.

Dans la troisième partie, vous devrez intégrer le travail réalisé dans les deux premières parties dans une architecture client-serveur permettant de simuler le fonctionnement réel d'un système informatique de bibliothèque.

Dans cette architecture, un programme principal jouera le rôle du serveur et représentera le bibliothécaire. Plusieurs copies du même programme joueront le rôle des clients pourront être exécutées simultanément afin de représenter différents utilisateurs de la bibliothèque. Les clients pourront envoyer des demandes au serveur afin d'effectuer différentes opérations. Le serveur devra recevoir ces demandes, les traiter à l'aide du système de gestion de bibliothèque, puis retourner une réponse indiquant le résultat de l'opération demandée.

Partie I : gestion de la bibliothèque

Dans cette partie, vous allez développer les différentes classes nécessaires à la gestion de la bibliothèque, notamment les livres, les utilisateurs, les emprunts, les réservations ainsi que les structures de données permettant de les gérer.

Règles de gestion

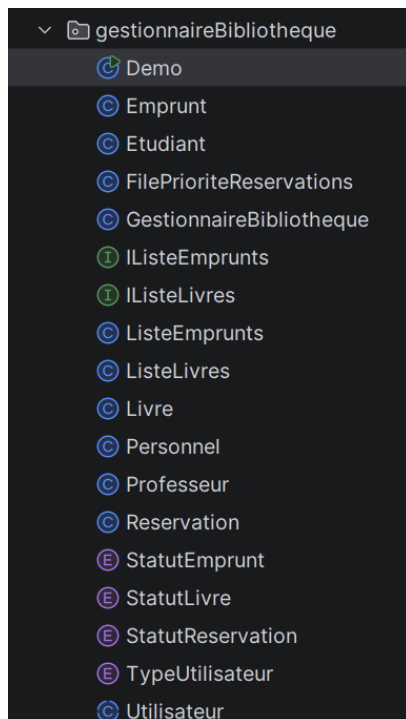
Dans cette application, chaque livre correspond à un seul exemplaire.

- Un même livre ne peut donc être emprunté que par un seul utilisateur à la fois. Lorsqu'un livre n'est pas disponible, un utilisateur peut effectuer une réservation. Les réservations sont placées dans une file de priorité afin de déterminer automatiquement quel utilisateur sera prioritaire lors du retour du livre.
- Lorsqu'un livre réservé est retourné, il est automatiquement attribué à l'utilisateur ayant la réservation prioritaire. Lorsqu'un livre possède au moins une réservation en attente, son statut devient RESERVE. Ce statut signifie qu'un livre est actuellement emprunté et possède au moins une réservation en attente.
- Un livre peut être dans l'un des états suivants : DISPONIBLE, EMPRUNTE, RESERVE ou PERDU.

- La durée maximale d'un emprunt est de 40 jours.
- Le nombre maximal d'emprunts autorisés dépend du type d'utilisateur. Un étudiant peut emprunter jusqu'à 5 livres, un membre du personnel jusqu'à 7 livres et un professeur jusqu'à 10 livres.
- Un utilisateur peut avoir au maximum 10 réservations en attente.
- Les réservations sont traitées selon l'ordre de priorité suivant : professeur, personnel puis étudiant. Pour deux utilisateurs ayant le même type, la réservation la plus ancienne est prioritaire.
- En cas de retard, une pénalité doit être calculée selon le type d'utilisateur : 0,25 \$ par jour pour un étudiant, 0,50 \$ pour un membre du personnel et 0,75 \$ pour un professeur.

Travail à réaliser

Vous devez créer et implémenter les classes et énumérations suivantes dans le package: *gestionnaireBibliotheque*



Tous les attributs des classes que vous créez doivent être privés. Les accesseurs (getters/setters) nécessaires au bon fonctionnement de l'application peuvent être ajoutés au besoin. Les noms des classes, des méthodes et des attributs demandés dans l'énoncé doivent être respectés.

Exercice 1 : Énumérations

Exercice 1.1

Implémenter l'énumération `TypeUtilisateur` ayant les labels :

ETUDIANT, PERSONNEL, PROFESSEUR

Cette énumération permettra d'indiquer la nature d'un utilisateur de la bibliothèque.

Exercice 1.2

Implémenter l'énumération StatutLivre ayant les labels :

DISPONIBLE, EMPRUNTE, RESERVE, PERDU

Cette énumération permettra d'indiquer l'état courant d'un livre.

Exercice 1.3

Implémenter l'énumération StatutEmprunt ayant les labels :

EN_COURS, RETOURNE, EN_RETARD, PERDU

Cette énumération permettra d'indiquer l'état d'un emprunt.

Exercice 1.4

Implémenter l'énumération StatutReservation ayant les labels :

EN_ATTENTE, ATTRIBUEE, ANNULEE

Cette énumération permettra d'indiquer l'état d'une réservation.

Classe Livre

Exercice 2 : Classe Livre

Exercice 2.1

Implémenter la classe Livre ayant les attributs suivants :

- id : un entier représentant l'identifiant unique du livre
- titre : une chaîne de caractères représentant le titre du livre
- auteur : une chaîne de caractères représentant l'auteur du livre
- categorie : une chaîne de caractères représentant la catégorie du livre
- statut de type StatutLivre : représente le statut courant du livre. Cet attribut doit être initialisé à DISPONIBLE

Exercice 2.2 :

Dans la classe Livre :

- Définir une constante nommée DUREE_MAX_EMPRUNT valant 40
- Implémenter une méthode statique nommée prochainID() qui retourne un numéro séquentiel commençant à 1
- Cette méthode servira à initialiser automatiquement l'identifiant des livres.
- Indication : utiliser un attribut statique dans la classe Livre.

Exercice 2.3 :

Dans la classe Livre, implémenter un constructeur permettant d'initialiser les attributs :

- titre
- auteur

- categorie
- L'identifiant doit être généré automatiquement à l'aide de la méthode `prochainID()`.
- Le statut du livre doit être initialisé à DISPONIBLE.

Exercice 2.4 :

Redéfinir la méthode `toString()` de la classe Livre afin d'afficher l'ensemble des attributs de la classe.

Exercice 2.6 :

Redéfinir la méthode `equals()` de la classe Livre.

Deux objets Livre sont considérés égaux s'ils possèdent le même identifiant.

Exercice 3 : Classe Emprunt

Exercice 3.1 :

Implémenter la classe Emprunt ayant les attributs suivants :

- id : un entier représentant l'identifiant unique de l'emprunt
- livre de type Livre : représente le livre emprunté
- idUtilisateur : un entier représentant l'identifiant de l'utilisateur ayant effectué l'emprunt
- jourEmprunt : un entier représentant le jour de l'emprunt
- jourRetourPrevu : un entier représentant le jour prévu du retour
- jourRetour : un entier représentant le jour réel du retour
- statut de type StatutEmprunt : représente le statut courant de l'emprunt
- L'attribut statut doit être initialisé à EN_COURS.

Exercice 3.2 :

Dans la classe Emprunt, implémenter une méthode statique nommée `prochainID()` permettant de générer automatiquement un identifiant unique pour chaque emprunt.

Exercice 3.3

Dans la classe Emprunt, implémenter un constructeur permettant d'initialiser :

- le livre emprunté ;
- l'identifiant de l'utilisateur ;
- le jour de l'emprunt.
- Le jour prévu du retour doit être calculé automatiquement à partir de la durée maximale d'emprunt.

Dans ce travail pratique, les dates sont représentées par des entiers. Chaque entier correspond à un numéro de jour dans la simulation. Par exemple, le jour 5 représente le cinquième jour de la simulation.

Ainsi, si un livre est emprunté au jour 5 et que la durée maximale d'un emprunt est de 40 jours, le jour prévu du retour sera donc le jour 45.

Exercice 3.4

Dans la classe Emprunt, implémenter la méthode :

```
public boolean estEnRetard(int jourActuel)
```

Cette méthode retourne true si le jour actuel dépasse le jour prévu du retour, retourne false sinon.

Exercice 3.5

Dans la classe Emprunt, implémenter la méthode :

```
public int calculerJoursRetard(int jourActuel)
```

Cette méthode retourne le nombre de jours de retard de l'emprunt.

Si l'emprunt n'est pas en retard, la méthode retourne 0.

Exercice 3.6 :

Dans la classe Emprunt, implémenter la méthode :

```
public void retourner(int jourRetour)
```

Cette méthode met à jour :

- le jour réel du retour ;
- le statut de l'emprunt.
- Le statut doit devenir :
 RETOURNE si le livre est retourné à temps ;
 EN_RETARD sinon.

Exercice 3.7 :

Redéfinir la méthode *toString()* de la classe Emprunt afin d'afficher les informations de l'emprunt.

Exercice4 : Classe Reservation**Exercice 4.1 :**

Implémenter la classe Reservation ayant les attributs suivants :

- id : un entier représentant l'identifiant unique de la réservation ;
- livre de type Livre : représente le livre réservé ;
- idUtilisateur : un entier représentant l'identifiant de l'utilisateur ayant fait la réservation ;
- typeUtilisateur de type TypeUtilisateur : représente le type de l'utilisateur ayant fait la réservation
- ordreReservation : un entier représentant l'ordre d'arrivée de la réservation ;
- statut de type StatutReservation : représente le statut courant de la réservation.
- L'attribut statut doit être initialisé à EN_ATTENTE.

Exercice 4.2 :

Dans la classe Reservation, implémenter une méthode statique nommée *prochainID()* permettant de générer automatiquement un identifiant unique pour chaque réservation.

Cette méthode doit retourner un numéro séquentiel commençant à 1.

Exercice 4.3 :

Dans la classe Reservation, implémenter une méthode statique nommée *prochainOrdreReservation()* permettant de générer automatiquement l'ordre d'arrivée des réservations.

Cette méthode doit retourner un numéro séquentiel commençant à 1.

Exercice 4.4 :

Dans la classe Reservation, implémenter un constructeur permettant d'initialiser :

- le livre réservé ;
- l'identifiant de l'utilisateur ;
- le type de l'utilisateur.
- L'identifiant de la réservation doit être généré automatiquement avec *prochainID()*.
- L'ordre de réservation doit être généré automatiquement avec *prochainOrdreReservation()*.
- Le statut de la réservation doit être initialisé à *EN_ATTENTE*.

Exercice 4.5 :

Définir l'ordre naturel des réservations en redéfinissant la méthode *compareTo*

La classe Reservation doit implémenter l'interface :

```
Comparable<Reservation>
```

L'ordre des réservations est défini comme suit :

- les réservations faites par les professeurs sont prioritaires ;
- les réservations faites par le personnel viennent ensuite ;
- les réservations faites par les étudiants viennent en dernier ;
- pour deux réservations faites par des utilisateurs du même type, la réservation la plus ancienne est prioritaire.

Ainsi, deux réservations ayant le même type d'utilisateur et le même ordre de réservation sont considérées comme égales selon ces critères.

Vous devez redéfinir la méthode :

```
public int compareTo(Reservation autre)
```

Cette méthode doit retourner :

- -1 si la réservation courante est prioritaire par rapport à autre ;
- 0 si les deux réservations sont équivalentes selon les critères de priorité ;
- 1 si la réservation courante est moins prioritaire que autre.

Exercice 4.6 :

Redéfinir la méthode *toString()* de la classe Reservation afin d'afficher les informations de la réservation, notamment :

- l'identifiant de la réservation ;
- l'identifiant du livre réservé ;
- l'identifiant de l'utilisateur ;

- le type d'utilisateur ;
- l'ordre de réservation ;
- le statut de la réservation.

Exercice 5 Classe FilePrioriteReservations

Exercice 5.1 :

Pour gérer les réservations, vous devez implémenter la classe FilePrioriteReservations. Cette classe servira à stocker toutes les réservations en attente de la bibliothèque dans une file de priorité.

La classe devra encapsuler un objet de type :

```
PriorityQueue<Reservation>
```

La priorité des réservations sera déterminée par l'ordre naturel défini dans la méthode compareTo() de la classe Reservation. Ainsi, les réservations seront automatiquement organisées selon le type d'utilisateur, puis selon leur ordre d'arrivée.

Cette file contiendra les réservations associées à tous les livres de la bibliothèque. Certaines méthodes permettront de manipuler la file de manière générale, tandis que la méthode retirerPourLivre(...) permettra de retirer uniquement la réservation prioritaire associée à un livre précis.

Exercice 5.2 :

Dans la classe FilePrioriteReservations, implémenter un constructeur qui initialise la file de priorité.

Exercice 5.3 :

Dans la classe FilePrioriteReservations, implémenter la méthode :

```
public boolean ajouter(Reservation reservation)
```

Cette méthode ajoute une réservation dans la file de priorité.

Elle retourne true si l'ajout a été effectué correctement.

Exercice 5.4 :

Dans la classe FilePrioriteReservations, implémenter la méthode :

```
public Reservation retirer()
```

Cette méthode retire et retourne la réservation la plus prioritaire.

Si la file est vide, elle retourne null.

Exercice 5.5 :

Dans la classe FilePrioriteReservations, implémenter la méthode :

```
public Reservation consulter()
```

Cette méthode retourne la réservation la plus prioritaire sans la retirer de la file.

Si la file est vide, elle retourne null.

Exercice 5.6 :

Dans la classe FilePrioriteReservations, implémenter la méthode :

```
public boolean estVide()
```

Cette méthode retourne true si la file ne contient aucune réservation, retourne false sinon.

Exercice 5.7 :

Dans la classe FilePrioriteReservations, implémenter la méthode :

```
public int taille()
```

Cette méthode retourne le nombre de réservations actuellement présentes dans la file.

Exercice 5.8 :

Redéfinir la méthode toString() de la classe FilePrioriteReservations.

Cette méthode doit retourner une chaîne de caractères contenant l'ensemble des réservations présentes dans la file.

Exercice 5.9

retrait de la réservation prioritaire d'un livre

Prototype :

```
public Reservation retirerPourLivre(int idLivre)
```

Cette méthode permet de retirer et retourner la réservation prioritaire associée à un livre donné.

Si aucune réservation n'existe pour ce livre, la méthode retourne null.

Exercice 6 Classe Utilisateur**Exercice 6.1 :**

Pour représenter les utilisateurs de la bibliothèque, implémenter la classe abstraite Utilisateur.

Cette classe aura les attributs suivants :

- id : un entier représentant l'identifiant unique de l'utilisateur ;
- nom : une chaîne de caractères représentant le nom de l'utilisateur ;
- empruntsEnCours : un objet de type ListeEmprunts contenant les emprunts actuellement en cours
- empruntsTerminees : un objet de type ListeEmprunts contenant les emprunts déjà retournés ;
- reservations : un objet de type FilePrioriteReservations contenant les réservations faites par l'utilisateur. cette structure sert uniquement à conserver les réservations de l'utilisateur. L'ordre de priorité des réservations est géré par la file globale de la bibliothèque.

Note : ListeEmprunts est une classe qui sera implémentée dans les exercices suivants.

Exercice 6.2 :

Dans la classe Utilisateur, définir la constante suivante :

```
public static final int MAX_RESERVATIONS = 10;
```

Cette constante représente le nombre maximal de réservations qu'un utilisateur peut avoir en attente.

Exercice 6.3 :

Dans la classe Utilisateur, implémenter un constructeur permettant d'initialiser :

- l'identifiant de l'utilisateur ;
- le nom de l'utilisateur.
- Les listes d'emprunts et la file de réservations doivent être initialisées dans le constructeur.

Exercice 6.4 :

Dans la classe Utilisateur, implémenter la méthode :

```
public void ajouterEmpruntEnCours(Emprunt emprunt)
```

Cette méthode ajoute un emprunt à la liste des emprunts en cours de l'utilisateur.

Exercice 6.5 :

Dans la classe Utilisateur, implémenter la méthode :

```
public void ajouterEmpruntTermine(Emprunt emprunt)
```

Cette méthode ajoute un emprunt à la liste des emprunts terminés de l'utilisateur.

Exercice 6.6 :

Dans la classe Utilisateur, implémenter la méthode :

```
public Emprunt supprimerEmpruntEnCours(int idEmprunt)
```

Cette méthode doit rechercher dans la liste des emprunts en cours de l'utilisateur l'emprunt correspondant à l'identifiant reçu en paramètre, puis le retirer de la liste.

Si l'emprunt est trouvé, la méthode retourne l'objet Emprunt supprimé. Si aucun emprunt ne correspond à l'identifiant donné, la méthode retourne null.

Exercice 6.7 :

Dans la classe Utilisateur, implémenter la méthode :

```
public Emprunt rechercherEmpruntEnCours(int idEmprunt)
```

Cette méthode doit rechercher dans la liste des emprunts en cours de l'utilisateur l'emprunt correspondant à l'identifiant reçu en paramètre.

Si un emprunt correspondant est trouvé, la méthode retourne l'objet Emprunt associé. Sinon, la méthode retourne null.

Exercice 6.8 : recherche d'un emprunt terminé

Dans la classe Utilisateur, implémenter la méthode :

```
public Emprunt rechercherEmpruntTermine(int idEmprunt)
```

Cette méthode recherche un emprunt terminé à partir de son identifiant.

Elle retourne l'objet Emprunt trouvé si l'emprunt existe dans la liste des emprunts terminés de l'utilisateur. Si aucun emprunt terminé ne correspond à l'identifiant donné, la méthode retourne null.

Cette méthode sera notamment utilisée lors du traitement de la commande PENALTY, car une pénalité peut être calculée autant sur un emprunt en cours que sur un emprunt déjà terminé.

Exercice 6.9 :

Dans la classe Utilisateur, implémenter la méthode :

```
public boolean aDesEmpruntsEnCours()
```

Cette méthode retourne true si l'utilisateur possède au moins un emprunt en cours. Elle retourne false sinon.

Exercice 6.10 :

Dans la classe Utilisateur, déclarer la méthode abstraite :

```
public abstract int nombreMaxEmprunts();
```

Cette méthode retournera le nombre maximal de livres qu'un utilisateur peut emprunter selon son type.

Exercice 6.11 :

Dans la classe Utilisateur, implémenter la méthode :

```
public boolean peutEmprunter()
```

Cette méthode retourne true si le nombre d'emprunts en cours de l'utilisateur est inférieur au nombre maximal d'emprunts autorisé pour son type. Elle retourne false sinon.

Le nombre maximal d'emprunts dépendra de la méthode abstraite nombreMaxEmprunts().

Exercice 6.12 :

Dans la classe Utilisateur, implémenter la méthode :

```
public boolean possedeReservationEnAttentePourLivre (int idLivre)
```

Cette méthode doit vérifier si l'utilisateur possède déjà une réservation en attente pour le livre correspondant à l'identifiant reçu en paramètre.

La méthode retourne true si une réservation en attente existe déjà pour ce livre, et false sinon.

Cette méthode sera notamment utilisée afin d'empêcher un utilisateur de réserver plusieurs fois le même livre.

Exercice 6.13 :

Dans la classe Utilisateur, implémenter la méthode :

```
public boolean ajouterReservation(Reservation reservation)
```

Cette méthode ajoute une réservation à la file des réservations de l'utilisateur si le nombre maximal de réservations n'est pas atteint. Elle retourne true si la réservation a été ajoutée et false sinon.

Exercice 6.14 :

Dans la classe Utilisateur, déclarer la méthode abstraite :

```
public abstract double calculerPenalite(int joursRetard);
```

Cette méthode calculera le montant de la pénalité selon le nombre de jours de retard. Le calcul dépendra du type d'utilisateur.

Exercice 6.15 :

Dans la classe Utilisateur, déclarer la méthode abstraite :

```
public abstract TypeUtilisateur getTypeUtilisateur();
```

Cette méthode retourne le type de l'utilisateur.

Les sous-classes Etudiant, Personnel et Professeur devront redéfinir cette méthode afin de retourner respectivement : ETUDIANT, PERSONNEL, PROFESSEUR

Exercice 9.16 :

Redéfinir la méthode toString() de la classe Utilisateur.

Cette méthode doit retourner une chaîne de caractères contenant :

- le type d'utilisateur ;
- l'identifiant ;
- le nom ;
- le nombre d'emprunts en cours ;
- le nombre d'emprunts terminés ;
- le nombre de réservations en attente.

Exercice 7: Classes Etudiant, Personnel et Professeur

Exercice 7.1 :

Implémenter la classe Etudiant.

Cette classe doit hériter de la classe abstraite Utilisateur.

Dans cette classe :

- le nombre maximal d'emprunts est de 5 .
- la pénalité est de 0.25 \$ par jour de retard.

Exercice 7.2 :

Implémenter la classe Personnel.

Cette classe doit hériter de la classe abstraite Utilisateur.

Dans cette classe :

- le nombre maximal d'emprunts est de 7 ;
- la pénalité est de 0.50 \$ par jour de retard.

Exercice 7.3 :

Implémenter la classe Professeur.

Cette classe doit hériter de la classe abstraite Utilisateur.

Dans cette classe :

- le nombre maximal d'emprunts est de 10 ;
- la pénalité est de 0.75 \$ par jour de retard.

Exercice 8 : Interfaces IListeLivres et IListeEmprunts

Exercice 8.1 :

Créer l'interface IListeLivres.

Cette interface devra déclarer les méthodes principales nécessaires à la gestion d'une liste de livres.

Parmi les méthodes à déclarer :

```
public boolean ajouter(Livre livre);
public Livre supprimer(int idLivre);
public Livre rechercher(int idLivre);
public boolean contient(int idLivre);
public int taille();
public boolean estVide();
```

Exercice 8.2 :

Créer l'interface IListeEmprunts.

Cette interface devra déclarer les méthodes principales nécessaires à la gestion d'une liste d'emprunts.

Parmi les méthodes à déclarer :

```
public boolean ajouter(Emprunt emprunt);
public Emprunt supprimer(int idEmprunt);
public Emprunt rechercher(int idEmprunt);
public boolean contient(int idEmprunt);
public int taille();
public boolean estVide();
```

Exercice 9 Classe ListeLivres

Exercice 9.1 :

Implémenter la classe ListeLivres.

Cette classe doit implémenter l'interface IListeLivres.

La classe ListeLivres devra encapsuler une structure de données de type :

```
ArrayList<Livre>
```

pour stocker les objets Livre.

Exercice 9.2 :

Dans la classe ListeLivres, implémenter un constructeur permettant d'initialiser la liste interne.

Exercice 9.3 :

Dans la classe ListeLivres, implémenter la méthode :

```
public boolean ajouter(Livre livre)
```

Cette méthode ajoute un livre dans la liste, retourne true si l'ajout a été effectué correctement.

Exercice 9.4 :

Dans la classe ListeLivres, implémenter la méthode :

```
public Livre supprimer(int idLivre)
```

Cette méthode doit rechercher dans la liste le livre correspondant à l'identifiant reçu en paramètre, puis le retirer de la liste. Si le livre est trouvé, la méthode retourne l'objet Livre supprimé. Si aucun livre ne correspond à l'identifiant donné, la méthode retourne null.

Exercice 9.5 :

Dans la classe ListeLivres, implémenter la méthode :

```
public Livre rechercher(int idLivre)
```

Cette méthode retourne le livre correspondant à l'identifiant reçu en paramètre. Si aucun livre ne correspond à cet identifiant, la méthode retourne null.

Exercice 9.6 :

Dans la classe ListeLivres, implémenter la méthode :

```
public boolean contient(int idLivre)
```

Cette méthode retourne true si un livre correspondant à l'identifiant existe dans la liste, retourne false sinon.

Exercice 9.7 :

Dans la classe ListeLivres, implémenter les méthodes :

```
public int taille()  
public boolean estVide()
```

Exercice 9.8 :

La classe ListeLivres doit implémenter l'interface :

```
Iterable<Livre>
```

Vous devez redéfinir la méthode :

```
public Iterator<Livre> iterator()
```

Cette méthode retourne un itérateur permettant de parcourir les livres de la liste.

Exercice 9.9 :

Redéfinir la méthode toString() de la classe ListeLivres.

Cette méthode doit retourner une chaîne de caractères contenant tous les livres présents dans la liste.

Exercice 10 Classe ListeEmprunts**Exercice 10.1 :**

Implémenter la classe ListeEmprunts.

Cette classe doit implémenter l'interface IListeEmprunts et utiliser une structure de données de type

```
ArrayList<Emprunt>
```

pour stocker les objets Emprunt.

Exercice 10.2 :

Dans la classe ListeEmprunts, implémenter un constructeur permettant d'initialiser la liste interne.

Exercice 10.3 :

Dans la classe ListeEmprunts, implémenter la méthode :

```
public boolean ajouter(Emprunt emprunt)
```

Cette méthode ajoute un emprunt dans la liste et retourne true si l'ajout a été effectué correctement.

Exercice 10.4 :

Dans la classe ListeEmprunts, implémenter la méthode :

```
public Emprunt supprimer(int idEmprunt)
```

Cette méthode retire de la liste l'emprunt correspondant à l'identifiant reçu en paramètre.

Elle retourne l'objet supprimé. Si aucun emprunt ne correspond à cet identifiant, la méthode retourne null.

Exercice 10.5 :

Dans la classe ListeEmprunts, implémenter la méthode :

```
public Emprunt rechercher(int idEmprunt)
```

Cette méthode retourne l'emprunt correspondant à l'identifiant reçu en paramètre. Si aucun emprunt ne correspond à cet identifiant, la méthode retourne null.

Exercice 10.6 :

Dans la classe ListeEmprunts, implémenter la méthode :

```
public boolean contient(int idEmprunt)
```

Cette méthode retourne true si un emprunt correspondant à l'identifiant existe dans la liste, retourne false sinon.

Exercice 10.7 :

Dans la classe ListeEmprunts, implémenter les méthodes :

```
public int taille()
public boolean estVide()
```

Exercice 10.8 :

La classe ListeEmprunts doit implémenter l'interface :

```
Iterable<Emprunt>
```

Vous devez redéfinir la méthode :

```
public Iterator<Emprunt> iterator()
```

Cette méthode retourne un itérateur permettant de parcourir les emprunts de la liste.

Exercice 10.9 :

Redéfinir la méthode toString() de la classe ListeEmprunts. Cette méthode doit retourner une chaîne de caractères contenant tous les emprunts présents dans la liste.

Exercice 11 Classe de démonstration et de vérification

Afin de faciliter la validation des différentes classes réalisées dans cette partie, vous devrez créer une classe Demo.

Cette classe permettra de tester progressivement le fonctionnement du système à l'aide d'affichages dans la console. Vous devrez y créer manuellement quelques objets nécessaires aux tests, par exemple quelques livres, utilisateurs, emprunts et réservations.

Vous pourrez ensuite utiliser des instructions d'affichage afin de vérifier le comportement des différentes méthodes programmées.

Cette classe pourra notamment être utilisée pour :

- afficher les informations d'un livre ;
- vérifier qu'un emprunt est correctement créé ;
- tester le calcul du retour prévu ;
- vérifier si un emprunt est en retard ;
- tester le calcul des pénalités ;
- vérifier l'ordre de priorité des réservations ;
- tester les différentes listes et structures de données.

```
Livre livre = new Livre("Java", "Deitel", "Informatique");
Etudiant etudiant = new Etudiant(101, "Amine");
Emprunt emprunt = new Emprunt(livre, etudiant.getId(), 5);
System.out.println(emprunt);

System.out.println(
    "Jours de retard : "
    + emprunt.calculerJoursRetard(50)
);
```


Partie II : traitement des commandes

Dans cette partie, vous allez implémenter le traitement des différentes commandes permettant d'utiliser le système de gestion de bibliothèque.

Les utilisateurs de la bibliothèque pourront interagir avec le système en envoyant des demandes à l'aide de commandes textuelles. Chaque commande représentera une opération à effectuer, par exemple emprunter un livre, retourner un emprunt ou effectuer une réservation.

Dans l'application, les commandes seront représentées sous forme de chaînes de caractères. Chaque commande contient :

- Un type de commande indiquant l'opération demandée, par exemple BORROW, RETURN ou RESERVE ;
- Des arguments contenant les informations nécessaires au traitement de la commande, par exemple un identifiant de livre ou un numéro de jour ;
- Un identifiant de connexion permettant d'identifier le client ayant envoyé la commande.

Le traitement des commandes sera centralisé dans la classe GestionnaireBibliotheque, qui représentera la classe principale de gestion du système.

Classe GestionnaireBibliotheque

La classe GestionnaireBibliotheque représente la classe principale du système de gestion de bibliothèque. Elle devra contenir les différentes structures de données nécessaires à la gestion des utilisateurs, des livres, des emprunts et des réservations.

La classe GestionnaireBibliotheque doit contenir les attributs privés suivants :

```
private Map<Integer, Utilisateur> utilisateurs;  
private Map<Integer, Utilisateur> utilisateursAuthentifies;  
private ListeLivres livres;  
private FilePrioriteReservations reservationsEnAttente;
```

Description des attributs

- L'attribut utilisateurs contient l'ensemble des utilisateurs enregistrés dans le système. La clé représente l'identifiant de l'utilisateur, tandis que la valeur correspond à l'objet Utilisateur associé.
- L'attribut utilisateursAuthentifies contient les utilisateurs actuellement connectés au système. La clé représente l'identifiant de connexion du client, tandis que la valeur correspond à l'utilisateur authentifié sur cette connexion.
- L'attribut livres contient la liste des livres présents dans la bibliothèque. Cette structure permettra notamment de rechercher, afficher, emprunter et réserver les livres.
- L'attribut reservationsEnAttente contient l'ensemble des réservations actuellement en attente dans la bibliothèque. Les réservations sont stockées dans une file de priorité afin de respecter automatiquement l'ordre de priorité défini dans le système : professeur, personnel, étudiant, puis ordre d'arrivée des réservations.

Méthode Principale :

Dans la classe `GestionnaireBibliotheque`, vous allez implémenter la méthode suivante :

```
public String traiterCommande(int idConnexion, String typeCommande, String argument)
```

Cette méthode reçoit l'identifiant de la connexion, le type de commande à traiter ainsi que les arguments associés à cette commande, elle doit identifier la commande reçue, appeler la méthode de traitement correspondante, puis retourner une réponse parmi les suivantes sous forme de chaîne de caractères.

Voici les types de commandes que peut recevoir la méthode:

- REGISTER <id utilisateur> <type utilisateur> <nom utilisateur> : enregistrer un nouvel utilisateur ;
- ID <id utilisateur> : s'authentifier auprès du serveur ;
- BOOKS : afficher la liste des livres de la bibliothèque.
- BORROW <id livre> <jour> : emprunter un livre disponible ;
- RETURN <id emprunt> <jour> : retourner un livre emprunté ;
- RESERVE <id livre> : réserver un livre non disponible
- INFO : afficher les emprunts en cours de l'utilisateur ;
- INFO <id emprunt> : afficher les informations d'un emprunt particulier ;
- PENALTY <id emprunt> <jour actuel> : calculer la pénalité associée à un emprunt ;
- EXIT : déconnecter le client de l'application.

Pour chaque commande reçue, la méthode retourne une réponse permettant d'indiquer si la commande a été traitée avec succès ou si une erreur est survenue.

Réponses en cas de succès

- REGISTERED <id utilisateur> <nom utilisateur> : le nouvel utilisateur a été enregistré.
- AUTHORIZED <id utilisateur> <nom utilisateur> : l'utilisateur a été authentifié.
- BORROW_OK <id emprunt> <id livre> <jour retour prévu> : le livre a été emprunté avec succès.
- RETURN_OK <id emprunt> <id livre> : le livre a été retourné avec succès.
- RESERVATION_OK <id reservation> <id livre> : la réservation a été enregistrée.
- RESERVATION_ASSIGNED <id emprunt> <id livre> : un livre retourné a été automatiquement attribué à l'utilisateur prioritaire.
- INFO <nb emprunts> {<id emprunt> <id livre> <jour emprunt> <jour retour prévu> <statut>}+ : le serveur retourne les emprunts en cours.
- PENALTY <montant> <jours retard> : le serveur retourne la pénalité calculée.
- END : le client a été correctement déconnecté.

Réponses en cas d'erreur

- AUTHENTICATION_ERROR : l'utilisateur n'est pas authentifié.
- ALREADY_AUTHENTICATED_ERROR : l'utilisateur est déjà authentifié avec cette connexion.
- TOO_MANY_CONNECTIONS_ERROR : l'utilisateur est déjà connecté ailleurs.
- BAD_ARGUMENT_ERROR : les paramètres de la commande sont invalides.
- COMMAND_ERROR : retournée lorsque la commande n'est pas reconnue.
- BOOK_NOT_FOUND_ERROR : le livre demandé n'existe pas.

- BOOK_NOT_AVAILABLE_ERROR : le livre n'est pas disponible pour l'emprunt.
- BORROW_LIMIT_ERROR : l'utilisateur a atteint sa limite d'emprunts.
- RESERVATION_LIMIT_ERROR : l'utilisateur a atteint sa limite de réservations.
- BAD_BORROW_ERROR : l'emprunt demandé n'existe pas pour cet utilisateur.
- BOOK_ALREADY_AVAILABLE_ERROR : le livre est déjà disponible et ne peut pas être réservé.

Afin de faciliter l'organisation du code, chaque commande devra être traitée par une méthode spécifique. Les exercices suivants consisteront donc à implémenter les différentes méthodes de traitement associées aux commandes de l'application.

Une fois ces méthodes complétées, vous pourrez finaliser la méthode traiterCommande(...) en appelant la méthode appropriée selon le type de commande reçu.

Exercice 1 : traitement de la commande ID

Dans la classe GestionnaireBibliotheque, implémenter la méthode :

```
public String traiterID(int idConnexion, String argument)
```

Cette méthode permet d'authentifier un utilisateur à partir de la commande ID <id utilisateur>. Par exemple, la commande **ID 102** indique que l'utilisateur ayant l'identifiant 102 souhaite s'authentifier. La méthode doit vérifier que l'identifiant reçu est valide, que l'utilisateur existe dans la liste des utilisateurs enregistrés, qu'il n'est pas déjà authentifié avec la connexion courante et qu'il n'est pas déjà connecté à partir d'un autre client.

Si toutes les vérifications sont respectées, l'utilisateur est associé à la connexion courante et ajouté à la liste des utilisateurs authentifiés. La méthode retourne alors AUTHORIZED <id utilisateur> <nom utilisateur>.

La méthode doit gérer les erreurs suivantes :

- Si l'identifiant reçu est invalide ou si aucun utilisateur ne correspond à cet identifiant, elle retourne BAD_ARGUMENT_ERROR.
- Si la connexion courante est déjà associée à un utilisateur authentifié, elle retourne ALREADY_AUTHENTICATED_ERROR.
- Si l'utilisateur est déjà connecté à partir d'un autre client, elle retourne TOO_MANY_CONNECTIONS_ERROR.

Exercice 2 : traitement de la commande BOOKS

Dans la classe GestionnaireBibliotheque, implémenter la méthode :

```
public String traiterBOOKS()
```

Cette méthode permet de retourner la liste des livres présents dans la bibliothèque à partir de la commande BOOKS. La commande BOOKS peut être utilisée sans authentification.

La méthode doit parcourir la liste des livres de la bibliothèque, récupérer les informations de chaque livre puis construire la réponse à retourner au client.

La réponse retournée doit respecter le format suivant :

BOOKS <nb livres> {<id livre> <titre> <auteur> <categorie> <statut>}+

où :

- <nb livres> représente le nombre total de livres retournés ;
- chaque bloc {...} représente les informations d'un livre.

Pour simplifier le protocole texte utilisé dans l'application, le titre, l'auteur et la catégorie d'un livre ne doivent pas contenir d'espaces.

Exemple de réponse possible :

BOOKS 2 {1 Java Deitel Informatique DISPONIBLE} {2 CleanCode Martin Informatique EMPRUNTE}

Exercice 3 : traitement de la commande BORROW

Dans la classe GestionnaireBibliotheque, implémenter la méthode

```
public String traiterBORROW(int idConnexion, String argument) {
```

Cette méthode permet à un utilisateur authentifié d'emprunter un livre disponible à l'aide de la commande BORROW <id livre> <jour>. Par exemple, la commande BORROW 5 12 indique que l'utilisateur souhaite emprunter le livre ayant l'identifiant 5 au jour 12.

La méthode doit d'abord vérifier que l'utilisateur est authentifié. Elle doit ensuite vérifier que le livre demandé existe, que son statut est DISPONIBLE et que l'utilisateur n'a pas atteint le nombre maximal d'emprunts autorisé pour son type.

Si toutes les validations sont respectées, la méthode doit créer l'objet Emprunt correspondant, l'ajouter à la liste des emprunts en cours de l'utilisateur, modifier le statut du livre à EMPRUNTE, puis retourner la réponse BORROW_OK <id emprunt> <id livre> <jour retour prévu>.

La méthode doit gérer les erreurs suivantes :

- si l'utilisateur n'est pas authentifié, elle retourne AUTHENTICATION_ERROR.
- Si les arguments reçus sont invalides, elle retourne BAD_ARGUMENT_ERROR.
- Si le livre demandé n'existe pas, elle retourne BOOK_NOT_FOUND_ERROR.
- Si le livre n'est pas disponible pour l'emprunt, elle retourne BOOK_NOT_AVAILABLE_ERROR.
- Si l'utilisateur a atteint sa limite d'emprunts, elle retourne BORROW_LIMIT_ERROR.

Exercice 4 : traitement de la commande RETURN

Dans la classe GestionnaireBibliotheque, implémenter la méthode

```
public String traiterRETURN(int idConnexion, String argument)
```

Cette méthode permet à un utilisateur authentifié de retourner un livre emprunté à l'aide de la commande RETURN <id emprunt> <jour>. Par exemple, la commande RETURN 12 45 indique que l'utilisateur retourne l'emprunt ayant l'identifiant 12 au jour 45.

La méthode doit vérifier que l'utilisateur est authentifié et que l'emprunt indiqué existe bien dans la liste de ses emprunts en cours. Si ces validations sont respectées, l'emprunt doit être retourné, son

statut doit être mis à jour, puis il doit être déplacé de la liste des emprunts en cours vers la liste des emprunts terminés.

Après le retour du livre, la méthode doit vérifier s'il existe une réservation en attente pour ce livre. Si aucune réservation n'existe, le livre redevient DISPONIBLE et la méthode retourne RETURN_OK <id emprunt> <id livre>.

Si une réservation prioritaire existe pour ce livre, elle doit être retirée de la file des réservations en attente, son statut doit devenir ATTRIBUEE, puis un nouvel emprunt doit être créé automatiquement pour l'utilisateur prioritaire. Ce nouvel emprunt doit utiliser le jour du retour comme jour d'emprunt et être ajouté aux emprunts en cours de cet utilisateur. Dans ce cas, la méthode retourne uniquement RESERVATION_ASSIGNED <id emprunt> <id livre>.

La méthode doit gérer les erreurs suivantes :

- si l'utilisateur n'est pas authentifié, elle retourne AUTHENTICATION_ERROR.
- Si les arguments reçus sont invalides, elle retourne BAD_ARGUMENT_ERROR.
- Si l'emprunt demandé n'existe pas ou n'appartient pas à l'utilisateur connecté, elle retourne BAD_BORROW_ERROR.

Exercice 5: traitement de la commande RESERVE

Dans la classe GestionnaireBibliotheque, implémenter la méthode :

```
public String traiterRESERVE(int idConnexion, String argument)
```

Cette méthode permet à un utilisateur authentifié de réserver un livre qui n'est pas disponible à l'aide de la commande RESERVE <id livre>. Par exemple, la commande RESERVE 8 indique que l'utilisateur souhaite réserver le livre ayant l'identifiant 8.

La méthode doit vérifier que l'utilisateur est authentifié, que le livre demandé existe et que ce livre n'est pas disponible. Elle doit également vérifier que l'utilisateur n'a pas atteint sa limite maximale de réservations et qu'il ne possède pas déjà une réservation en attente pour ce même livre.

Si toutes les validations sont respectées, la méthode doit créer l'objet Reservation correspondant, l'ajouter à la file de priorité des réservations, l'ajouter aux réservations de l'utilisateur, puis modifier le statut du livre à RESERVE.

En cas de succès, la méthode retourne RESERVATION_OK <id reservation> <id livre>.

La méthode doit gérer les erreurs suivantes :

- si l'utilisateur n'est pas authentifié, elle retourne AUTHENTICATION_ERROR.
- Si les arguments reçus sont invalides, elle retourne BAD_ARGUMENT_ERROR.
- Si le livre demandé n'existe pas, elle retourne BOOK_NOT_FOUND_ERROR.
- Si le livre est déjà disponible, elle retourne BOOK_ALREADY_AVAILABLE_ERROR.
- Si l'utilisateur a atteint sa limite de réservations, elle retourne RESERVATION_LIMIT_ERROR.
- Si l'utilisateur possède déjà une réservation en attente pour ce livre, elle retourne BAD_ARGUMENT_ERROR.

Exercice 6 : traitement de la commande INFO

Dans la classe GestionnaireBibliotheque, implémenter la méthode :

```
public String traiterINFO(int idConnexion, String argument)
```

Cette méthode permet à un utilisateur authentifié de consulter ses emprunts en cours à l'aide de la commande INFO.

La commande peut être utilisée de deux façons.

- La commande INFO permet d'afficher tous les emprunts en cours de l'utilisateur connecté.
- La commande INFO <id emprunt> permet d'afficher les informations d'un emprunt en cours précis.

La méthode doit vérifier que l'utilisateur est authentifié, déterminer si un identifiant d'emprunt a été fourni, puis construire la réponse à retourner à partir des emprunts en cours de l'utilisateur. Les emprunts terminés ne doivent pas être affichés par cette commande.

-Pour la commande INFO, la méthode doit retourner une réponse de la forme :

INFO <nb emprunts> {<id emprunt> <id livre> <jour emprunt> <jour retour prévu> <statut>}+.

Exemple : INFO 2 {1 5 10 50 EN_COURS} {2 3 12 52 EN_COURS}

-Pour la commande INFO <id emprunt>, la méthode doit rechercher l'emprunt demandé uniquement parmi les emprunts en cours de l'utilisateur connecté. Si l'emprunt est trouvé, elle retourne une réponse contenant un seul emprunt, sous la forme INFO 1 {<id emprunt> <id livre> <jour emprunt> <jour retour prévu> <statut>}.

Exemple : INFO 1 {12 5 10 50 EN_COURS}

La méthode doit gérer les erreurs suivantes :

- si l'utilisateur n'est pas authentifié, elle retourne AUTHENTICATION_ERROR.
- Si les arguments reçus sont invalides, elle retourne BAD_ARGUMENT_ERROR.
- Si l'emprunt demandé n'existe pas ou n'appartient pas à l'utilisateur connecté, elle retourne BAD_BORROW_ERROR.

Exercice 7: traitement de la commande PENALTY

Dans la classe GestionnaireBibliotheque, implémenter la méthode

```
public String traiterPENALTY(int idConnexion, String argument)
```

Cette méthode permet à un utilisateur authentifié de consulter la pénalité associée à l'un de ses emprunts à l'aide de la commande PENALTY <id emprunt> <jour actuel>. Par exemple, la commande

PENALTY 15 52 indique que l'utilisateur souhaite calculer la pénalité de l'emprunt ayant l'identifiant 15 au jour 52.

La méthode doit vérifier que l'utilisateur est authentifié, que l'emprunt existe et qu'il appartient bien à l'utilisateur connecté. Contrairement à la commande INFO, la recherche de l'emprunt doit être effectuée autant dans les emprunts en cours que dans les emprunts terminés de l'utilisateur.

Si l'emprunt est trouvé, la méthode doit calculer le nombre de jours de retard, puis calculer le montant de la pénalité selon le type de l'utilisateur. Elle retourne ensuite la réponse PENALTY <montant> <jours retard>.

Le montant de la pénalité doit être affiché avec deux décimales et utiliser le point comme séparateur décimal. Si l'emprunt n'est pas en retard, le montant retourné doit être 0.00 et le nombre de jours de retard doit être 0.

La méthode doit gérer les erreurs suivantes :

- Si l'utilisateur n'est pas authentifié, elle retourne AUTHENTICATION_ERROR.
- Si les arguments reçus sont invalides, elle retourne BAD_ARGUMENT_ERROR.
- Si l'emprunt demandé n'existe pas ou n'appartient pas à l'utilisateur connecté, elle retourne BAD_BORROW_ERROR.

Exercice 8 : traitement de la commande REGISTER

Dans la classe GestionnaireBibliotheque, implémenter la méthode

```
public String traiterREGISTER(String argument) {
```

Cette méthode permet d'enregistrer un nouvel utilisateur à l'aide de la commande

REGISTER <id utilisateur> <type utilisateur> <nom utilisateur>.

Par exemple, la commande *REGISTER 120 ETUDIANT Sara* indique qu'un nouvel utilisateur nommé Sara souhaite être enregistré dans le système avec l'identifiant 120 et le type ETUDIANT.

La méthode doit récupérer les paramètres de la commande, vérifier qu'ils sont valides puis s'assurer que l'identifiant utilisateur n'est pas déjà utilisé dans le système.

Si les validations sont respectées, la méthode doit créer l'objet utilisateur correspondant selon le type demandé (ETUDIANT, PERSONNEL ou PROFESSEUR), ajouter cet utilisateur dans la liste des utilisateurs enregistrés, puis retourner la réponse REGISTERED <id utilisateur> <nom utilisateur>.

La méthode doit gérer les erreurs suivantes :

- Si les arguments reçus sont invalides, si l'identifiant est déjà utilisé ou si le type d'utilisateur est invalide, elle retourne BAD_ARGUMENT_ERROR.

Exercice 9 : traitement de la commande EXIT

Dans la classe GestionnaireBibliotheque, implémenter la méthode :

```
public String traiterEXIT(int idConnexion)
```

Cette méthode permet à un utilisateur de quitter l'application à l'aide de la commande EXIT.

La méthode doit vérifier que la connexion courante est associée à un utilisateur authentifié. Si c'est le cas, l'utilisateur doit être retiré de la liste des utilisateurs authentifiés puis la méthode doit retourner la réponse END.

Si aucun utilisateur n'est authentifié sur cette connexion, la méthode retourne AUTHENTICATION_ERROR.

Important :

Avant de commencer l'intégration client-serveur de la partie III, il est essentiel de s'assurer que toutes les commandes fonctionnent correctement dans la console.

Il est fortement recommandé de tester chaque méthode dès son implémentation en vérifiant :

- les cas de succès ;
- les cas d'erreur ;
- les formats exacts des réponses retournées.

Une validation progressive du fonctionnement du système facilitera grandement l'intégration dans l'architecture client-serveur de la partie III.

Partie III — Intégration dans une architecture client-serveur

Dans les deux premières parties du travail pratique, vous avez développé les classes nécessaires à la gestion d'une bibliothèque ainsi que le traitement des différentes commandes permettant d'utiliser cette bibliothèque.

Dans cette troisième partie, nous allons intégrer ce travail dans une architecture client-serveur déjà fonctionnelle.

L'objectif de cette partie est de simuler le fonctionnement réel d'une application informatique où plusieurs utilisateurs peuvent utiliser la bibliothèque en même temps à partir de différents postes.

Principe général de l'architecture

L'architecture contiendra deux projets:

- Un projet serveur : représente l'application du bibliothécaire, Il contiendra les livres, les utilisateurs, les emprunts, les réservations ainsi que toute la logique de gestion de la bibliothèque
- Un projet client : Le projet client représente l'application utilisée par les utilisateurs de la bibliothèque, à partir de cette application, un utilisateur pourra envoyer différentes commandes afin d'interagir avec le système. Cette application est déjà entièrement fournie et ne doit pas être modifiée dans le cadre de ce travail.

Exemple de scénario de fonctionnement :

L'application serveur doit d'abord être démarrée. Une fois lancée, elle restera en attente des connexions provenant des applications clientes.

Dès qu'une application cliente est démarrée, elle se connecte automatiquement au serveur à l'aide des mécanismes de communication déjà implémentés dans le projet fourni pour cette troisième partie.

Une fois connectée, l'application cliente pourra envoyer différentes commandes au serveur. Par exemple, un utilisateur pourra écrire : BOOKS

Le serveur recevra alors cette commande, appellera les méthodes de traitement programmées dans la partie II, puis retournera la réponse appropriée au client. Dans cet exemple, le serveur retournera la liste des livres de la bibliothèque.

Tout le mécanisme de connexion, de communication et d'échange des messages entre le client et le serveur est déjà entièrement fonctionnel dans le projet fourni. Vous n'avez donc pas à modifier ni à comprendre le fonctionnement interne des classes réseau fournies.

Fonctionnement du code fourni

Le code fourni contient déjà toute l'architecture nécessaire pour permettre à plusieurs applications de communiquer entre elles.

Dans la version de départ fournis:

1. le client envoie une commande au serveur ;
2. le serveur reçoit cette commande ;
3. le serveur transforme la commande en majuscules !
4. puis retourne le résultat au client.

Exemple : le client ecris hello, il recevra : HELLO

Cette conversion en majuscule sert uniquement à vérifier que la communication entre le client et le serveur fonctionne correctement.

Dans cette partie, votre travail consistera à remplacer la conversion t par le système de gestion de bibliothèque développé dans les parties précédentes. Autrement dit , le serveur devra agir principalement comme intermédiaire entre les clients et la classe *GestionnaireBibliotheque*.

Réception des commandes : les événements

Lorsqu'un client envoie une commande, celle-ci est reçue par le serveur sous forme d'un objet : de type *Evenement* . Cet objet représente une demande reçue par l'application et contient :

- la connexion associée au client ayant envoyé la commande ;
- le type de commande ;
- les arguments associés à cette commande.

Chaque client connecté possède une connexion différente permettant au serveur d'identifier l'utilisateur ayant envoyé la commande et de lui retourner la bonne réponse.

Par exemple, si un utilisateur envoie : BORROW 5 12

l'événement contiendra notamment :

- le type de commande : BORROW
- les arguments : 5 12
- la connexion associée au client ayant envoyé la commande

Ces informations permettront ensuite d'appeler les méthodes appropriées de la classe : *GestionnaireBibliotheque*

Etapes d' Intégration du gestionnaire de bibliothèque

1. Ajouter dans le projet serveur toutes les classes réalisées dans les parties I et II dans le package : *gestionnaireBibliotheque*
2. S'assurer que toutes les classes compilent correctement dans le projet serveur.
3. Ouvrir la classe *GestionnaireEvenementServeur* : Cette classe est responsable du traitement des événements reçus par le serveur.



4. Dans cette classe, repérer la section du code où la commande reçue est actuellement convertie en majuscules puis retournée au client.
5. Créer dans cette classe un objet de type : *GestionnaireBibliotheque* qui permettra d'utiliser le système de gestion de bibliothèque.
6. Remplacer le traitement actuel par un appel à la méthode : *traiterCommande* de la classe : *GestionnaireBibliotheque*

Le traitement général devra être le suivant :

1. Récupérer un identifiant unique associé à la connexion du client.

Par exemple : `int idConnexion = System.identityHashCode(eventement.getConnexion());`

2. Récupérer le type de commande contenu dans l'événement : `(eventement.getType(); )`
3. Récupérer les arguments associés à la commande : `( eventement.getArgument(); )`
4. Transmettre ces informations au gestionnaire de bibliothèque en appelant la méthode `traiterCommande` :
5. Retourner la réponse produite au client.

Tester l'architecture

Afin de faciliter les tests de l'application, vous pouvez ajouter manuellement quelques livres et utilisateurs directement dans le constructeur de la classe *GestionnaireBibliotheque*.

Dans une application réelle, les livres et les utilisateurs seraient généralement déjà enregistrés dans une base de données ou dans des fichiers de sauvegarde. Cependant, dans le cadre de ce travail pratique, nous allons simplement créer quelques objets de test dans le constructeur afin que le serveur possède déjà des données lors de son démarrage.

Ainsi, dès que le serveur sera lancé, il sera possible de tester immédiatement les différentes commandes de l'application, par exemple BOOKS, ID, BORROW, RETURN ou INFO.

Exemple :

```
// AJOUTER UN LIVRE DE TEST
livres.ajouter( new Livre( titre: "Java", auteur: "Deitel", categorie: "Informatique") );
// AJOUTER UN UTILISATEUR DE TEST
utilisateurs.put( 101, new Etudiant( id: 101, nom: "Amine") );
```

Exécution de plusieurs clients

Afin de bien vérifier le fonctionnement des différentes commandes et de simuler une utilisation plus réaliste de l'application, il sera nécessaire d'exécuter plusieurs clients simultanément.

Pour cela, vous devrez lancer plusieurs instances de l'application cliente.

Sous IntelliJ IDEA, il faut d'abord modifier la configuration d'exécution du projet client.

Ensuite, dans la fenêtre de configuration : ouvrir le menu Modify options ; et activer l'option : Allow multiple instances.

Cette option permettra d'exécuter plusieurs copies de l'application cliente simultanément afin de simuler plusieurs utilisateurs connectés à la bibliothèque.

Remise du travail

La remise du travail se fera en deux étapes :

- Une première remise devra être effectuée deux semaines après la présentation du TP. Cette remise devra contenir les parties I et II du travail pratique ainsi qu'une classe Demo permettant de tester le fonctionnement des différentes classes et commandes dans la console.
- Une deuxième remise devra être effectuée à la quatrième semaine suivant la présentation du TP. Cette remise devra contenir uniquement le projet serveur complet incluant l'intégration client-serveur de la partie III. Prendre note qu'il ne faut pas modifier les classes du noyau réseau fournies.

Pour chaque remise, les consignes suivantes doivent être respectées :

- Respecter les noms des classes, méthodes et attributs demandés ;
- S'assurer que le projet compile et s'exécute correctement avant la remise ;